

Rwppport du projet TaskFlow API

Dans le cadre de ce projet, j'ai développé une API REST nommée **TaskFlow** avec **Django** et **Django REST Framework**. L'objectif était de créer une application permettant de gérer des membres, des équipes, des projets, des tâches et des commentaires.

Pour réaliser ce projet, je me suis appuyé sur les notions étudiées dans le cours **Développez votre site Web avec le framework Django**, où un site de gestion des **Crêpes bretonnes** est développé progressivement. Même si ce projet est différent, plusieurs concepts présentés dans le cours ont été réutilisés et adaptés à une API REST.

Le projet repose sur cinq modèles :

- **Member**
- **Team**
- **Project**
- **Task**
- **Comment**

Le modèle **Member** représente un membre de l'application. Chaque membre possède un nom et une adresse e-mail.

Le modèle **Team** représente une équipe. Une équipe possède un nom, une description, une date de création ainsi qu'une liste de membres.

La relation entre **Member** et **Team** est une relation **ManyToMany**. Le fonctionnement de cette relation est présenté dans le chapitre consacré aux relations entre modèles, où le livre explique qu'un **ManyToManyField** permet d'associer plusieurs objets entre eux et qu'une table intermédiaire est créée automatiquement lorsque cela est nécessaire (chapitre sur les relations entre modèles, **pages 77 à 79 environ**).

Le modèle **Project** est relié à une équipe grâce à une **ForeignKey**. Cette relation permet à plusieurs projets d'appartenir à une même équipe. Le principe des **ForeignKey** est présenté dans le chapitre sur les modèles relationnels (**pages 74 à 76 environ**).

Le modèle **Task** est relié à un projet et peut être affecté à un membre. Il contient également un statut, une priorité et une date d'échéance.

Enfin, le modèle **Comment** est associé à une tâche et à son auteur afin de permettre le suivi des différentes tâches.

L'ensemble de ces relations permet d'obtenir une structure cohérente entre les différentes entités du projet.

Comme présenté dans le cours, chaque modèle est déclaré dans le fichier **models.py**, puis les modifications sont appliquées grâce aux commandes :

```
python manage.py makemigrations
python manage.py migrate
```

Ces commandes sont expliquées dans le chapitre consacré aux modèles et aux migrations (**pages 65 à 67**).

Le projet utilise également l'administration Django afin de vérifier rapidement les données enregistrées dans la base de données. Le livre explique que l'administration est générée automatiquement par Django et peut être personnalisée dans le fichier **admin.py** (chapitre **II.4 L'administration, pages 86 à 93**).

Pour exposer les données sous forme d'API, j'ai utilisé les serializers et les ViewSets de Django REST Framework. Cette partie n'est pas développée dans le projet *Crêpes bretonnes*, mais elle a été réalisée en suivant la documentation officielle de Django REST Framework, tout en conservant l'organisation générale du projet Django présentée dans le cours.

validation

La principale validation mise en place concerne l'affectation des tâches.

Lorsqu'une tâche est créée, le membre sélectionné doit obligatoirement appartenir à l'équipe propriétaire du projet.

Si cette condition n'est pas respectée, l'API renvoie une erreur **400 (Bad Request)** avec un message expliquant que le membre doit appartenir à l'équipe du projet.

Cette validation a été implémentée dans le **TaskSerializer**, afin d'empêcher l'enregistrement de données incohérentes.

Les statuts et les priorités sont également limités grâce aux **TextChoices** définis dans les modèles. Ainsi, seules les valeurs prévues peuvent être enregistrées.

Pagination et filtrage

La liste des tâches utilise une **pagination** afin d'éviter de renvoyer toutes les données en une seule réponse.

Pour cela, une classe de pagination personnalisée a été créée dans le fichier **pagination.py**.

Le projet permet également de filtrer les tâches selon plusieurs critères, notamment :

- le statut ;
- la priorité ;
- le projet ;
- le membre assigné.

Ces filtres peuvent être combinés afin d'obtenir uniquement les tâches correspondant aux critères recherchés.

Le principe de la pagination est également présenté dans le support de cours (**chapitre IV.4 – Pagination, page 237**), même si son implémentation dans ce projet repose sur Django REST Framework.

Tests réalisés

Des tests unitaires ont été écrits afin de vérifier les principales fonctionnalités de l'application.

Les tests portent notamment sur :

- la création d'un membre ;
- la création d'une équipe ;
- l'ajout d'un membre dans une équipe ;
- la validation empêchant d'affecter une tâche à un membre qui n'appartient pas à l'équipe du projet.

Le livre présente l'intérêt des tests unitaires ainsi que leur mise en œuvre dans le chapitre **IV.6 Les tests unitaires (pages 253 à 257)**. Cette partie m'a servi de base pour comprendre le fonctionnement des tests dans Django.

Difficultés rencontrées

La première difficulté rencontrée concernait les relations entre les modèles.

Au début, je ne savais pas toujours quand utiliser une **ForeignKey** ou une **ManyToMany**. Après avoir repris les exemples du cours et réalisé plusieurs essais, j'ai mieux compris le rôle de chaque relation.

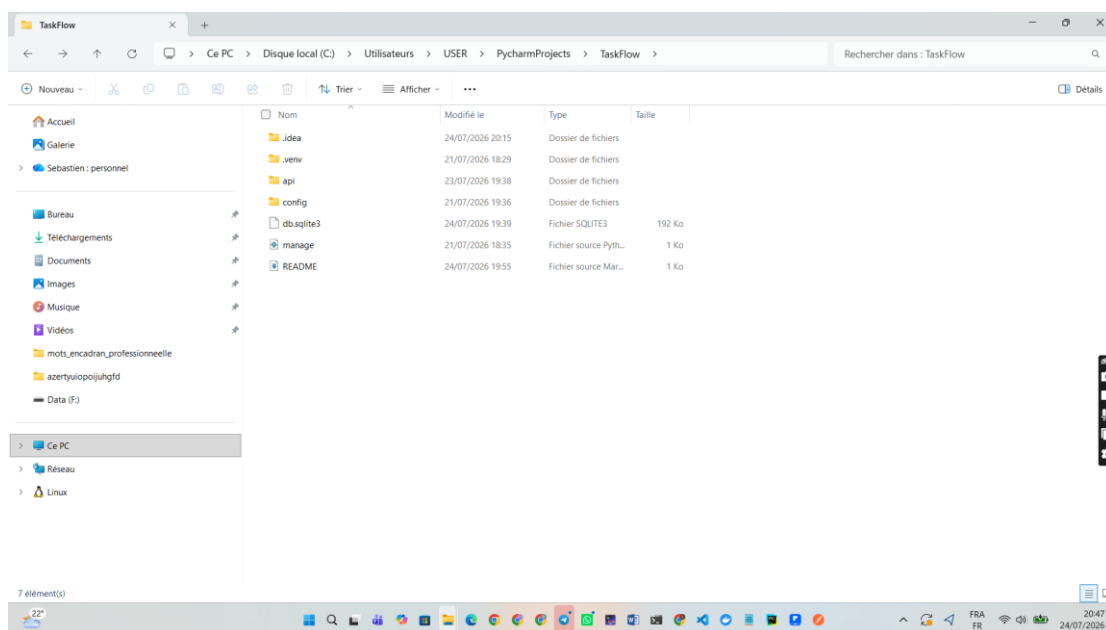
La deuxième difficulté concernait la validation métier. Au départ, une tâche pouvait être affectée à n'importe quel membre. Après plusieurs tests, cette vérification a été ajoutée dans le serializer afin d'empêcher cette situation.

J'ai également rencontré quelques difficultés lors de l'écriture des tests unitaires. Certains tests échouaient parce que les données créées ne respectaient pas les relations entre les modèles. Après correction des données de test, tous les tests ont été exécutés avec succès.

Enfin, lors des essais avec Postman, je pensais que les endpoints ne fonctionnaient pas. Après vérification, le problème venait de l'utilisation de la version Web de Postman, qui ne permettait pas d'accéder au serveur local. Une fois la version adaptée utilisée, les différents endpoints ont pu être testés correctement.

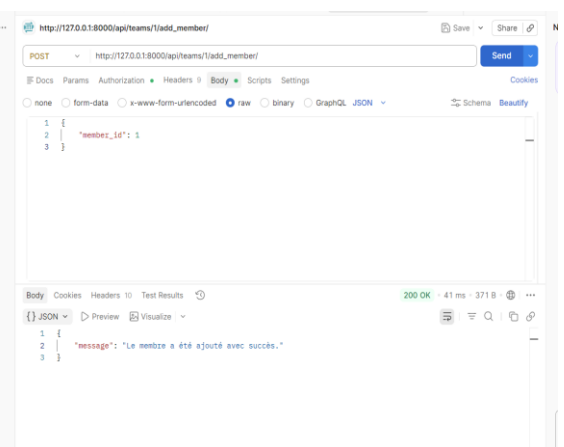
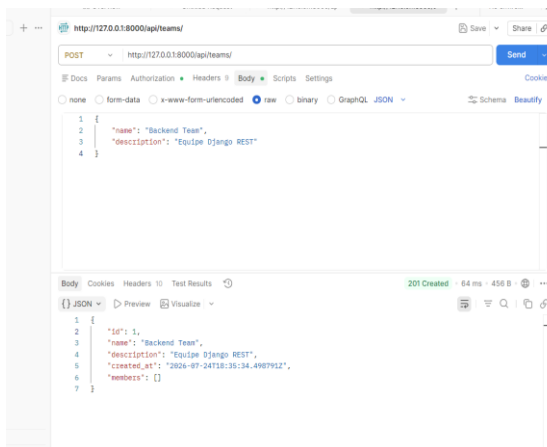
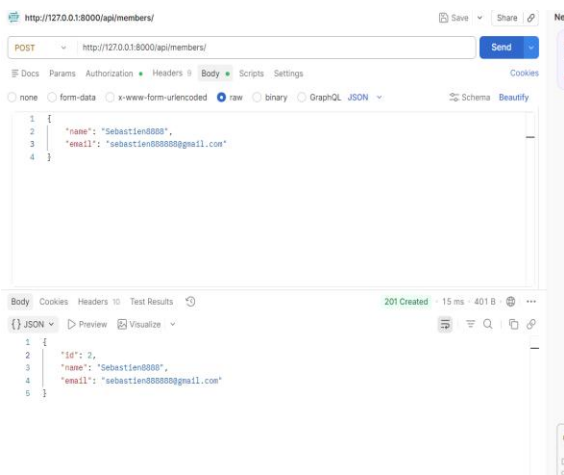
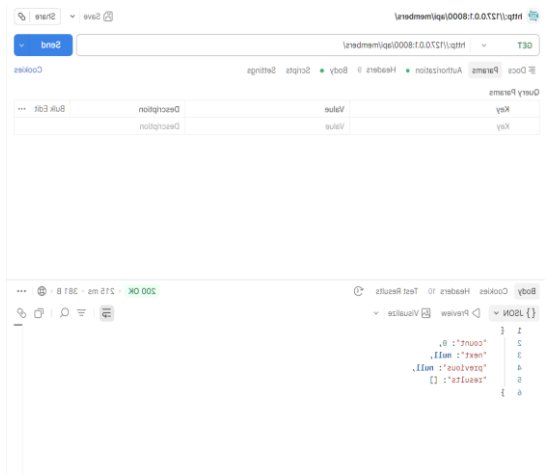
Time lapse : du 21 juillet au 24 juillet

☐ Nom	Modifié le	Type	Taille
.idea	24/07/2026 20:15	Dossier de fichiers	
.venv	21/07/2026 18:29	Dossier de fichiers	
api	23/07/2026 19:38	Dossier de fichiers	
config	21/07/2026 19:36	Dossier de fichiers	
db.sqlite3	24/07/2026 19:39	Fichier SQLITE3	192 Ko
manage	21/07/2026 18:35	Fichier source Pyth...	1 Ko
README	24/07/2026 19:55	Fichier source Mar...	1 Ko



Présentation des endpoints fonctionnels

Les captures d'écran suivantes présentent les différents endpoints testés avec **Postman**. Elles montrent le bon fonctionnement des opérations de création, de consultation, de modification et de suppression des données ainsi que des fonctionnalités développées dans l'API.



Four screenshots of a REST client interface showing API requests and responses for a task management system.

Top Left: GET /api/teams/1

Request: GET http://127.0.0.1:8000/api/teams/1

Response (200 OK):

```
{
  "name": "Backend Team",
  "description": "Koupe Django REST",
  "created_at": "2026-07-24T18:35:34.498791Z",
  "members": [
    {
      "id": 1
    }
  ]
}
```

Top Right: POST /api/projects/

Request: POST http://127.0.0.1:8000/api/projects/

Request Body:

```
{
  "title": "taskFlow",
  "description": "Projet de gestion des tâches",
  "status": "ACTIVE",
  "team": 1
}
```

Response (201 Created):

```
{
  "id": 1,
  "title": "taskFlow",
  "description": "Projet de gestion des tâches",
  "status": "ACTIVE",
  "team": 1
}
```

Bottom Left: POST /api/tasks/

Request: POST http://127.0.0.1:8000/api/tasks/

Request Body:

```
{
  "title": "Créer les modèles",
  "description": "Créer les modèles Django",
  "status": "TODO",
  "priority": "HIGH",
  "due_date": "2026-08-01",
  "project": 1,
  "assignee": 1
}
```

Response (201 Created):

```
{
  "id": 1,
  "title": "Créer les modèles",
  "description": "Créer les modèles Django",
  "status": "TODO",
  "priority": "HIGH",
  "due_date": "2026-08-01",
  "project": 1,
  "assignee": 1
}
```

Bottom Right: POST /api/comments/

Request: POST http://127.0.0.1:8000/api/comments/

Request Body:

```
{
  "content": "Très bon travail.",
  "author": 1,
  "task": 1
}
```

Response (201 Created):

```
{
  "id": 1,
  "content": "Très bon travail.",
  "created_at": "2026-07-24T18:39:42.788142Z",
  "author": 1,
  "task": 1
}
```

